Since 1983

MIT-WPU

॥ विद्यार्थिनिर्धुवं सुखम् ॥

Dr. Vishwanath Karad

MIT WORLD PEACE UNIVERSITY | PUNE

TECHNOLOGY, RESEARCH, SOCIAL INNOVATION & PARTNERSHIPS

Department of Computer Engineering and Technology

MINI PROJECT REPORT

Computer Networks Laboratory

(CSE20110)

Laboratory Continuous Assessment 3

NetRescue: Self – Healing Networks

Submitted by

Sr. No.	Student Name	PRN
1	Manjiri Sakore	1262242414
2	Rishita Kumar	1262242408
3	Aashnaa Panda	1262241937
4	Tanushree Doddamani	1262241836

Group No. 14 Panel K Batch 02

Third Year B.Tech. Computer Engineering and Technology

Under the guidance of

Dr. Uma Pujeri

Academic Year 2026–2027

Submission Date 19/10/2026

Since 1983

MIT-WPU
॥ विद्यावारिधिर्पुत्र पुजा ॥

Dr. Vishwanath Karad
MIT WORLD PEACE UNIVERSITY | PUNE
TECHNOLOGY, RESEARCH, SOCIAL INNOVATION & PARTNERSHIPS

Department of Computer Engineering and Technology

CERTIFICATE

This is to certify that the mini-project entitled "NetRescue: Network Resilience, Multipath Transfer and Diagnostics" has been carried out by the following students of Third Year B.Tech. Computer Science and Engineering, Panel [PANEL], Batch [BATCH], Group No. [GROUP NUMBER], under my guidance during the academic year 2026–2027.

Student Name	PRN
Manjiri Sakore	1262242414
Rishita Kumar	1262242408
Aashnaa Panda	1262241937
Tanushree Doddamani	1262241836

The report is submitted towards the mini-project component of Laboratory Continuous Assessment 3 for the Computer Networks Laboratory course.

Faculty Signature _____

[FACULTY NAME] Dr. Uma Pujeri
Project Guide and Course Faculty

Date 26.10.26 Place: Pune

Acknowledgement

We sincerely thank our faculty guide and the Department of Computer Engineering and Technology for their guidance and support throughout the development of NetRescue. We acknowledge the technical resources, development tools, and documentation that supported implementation, testing, and evaluation. We also thank our team members for their collaboration and contributions.

Group No.14

Abstract

Network failures, packet loss, latency, and unreliable data transfers can affect network performance and data integrity. NetRescue is a network resilience and diagnostics project designed to study failure detection, recovery behavior, multipath file transfer, and network performance through controlled experiments. The project aims to evaluate recovery mechanisms, verify transferred-file integrity, and provide useful network diagnostic measurements. NetRescue implements three main features. First, hysteresis-based failure detection uses configurable thresholds to manage network state transitions and reduce unnecessary switching caused by temporary instability. Second, multipath file transfer divides files into chunks, distributes them across multiple logical paths, reassembles them, and uses SHA-256 hashing to check data integrity. Third, network diagnostics and benchmarking support connectivity checks, DNS resolution, route tracing, latency and packet-loss measurements, and performance comparisons. The system uses Python and FastAPI for backend services and React with TypeScript for the frontend. Testing combines host-level diagnostics with controlled, application-level simulations. The project demonstrates recovery logic, file reassembly and integrity verification, and network performance analysis. The logical-path simulations do not establish performance gains over real physical multipath connections. NetRescue provides a foundation for further work on adaptive scheduling and real-world network integration.

Keywords: Network Resilience, Hysteresis-Based Recovery, Multipath File Transfer, Network Diagnostics

Table of Contents

Sr. No.	Section No.	Table of Contents
1	Chapter 1	Introduction
2	1.1	Project Objectives and Verification Methods
3	Chapter 2	System Requirements and Design
4	2.1	Tools and System Requirements
5	2.2	Addressing and Protocol Selection
6	Chapter 3	Implementation
7	Chapter 4	Testing and Results
8	4.1	Test Plan
9	4.2	Objective Achievement
10	Chapter 5	Troubleshooting and Discussion
11	5.1	Troubleshooting Summary
12	Chapter 6	Conclusion and Future Scope
13	—	References
14	—	Appendix

List of Tables

Table No.	Objective Achieved	Page no
Table 1.1	Project Objectives and Verification Methods	07
Table 2.1	Tools and System Requirements	09
Table 2.2	Addressing and Protocol Selection	10
Table 4.1	Test Plan	14
Table 4.2	Troubleshooting Summary	16

1. Introduction and Project Objectives

1.1 Introduction and Background

Computer networks rely on reliable communication, route availability, and timely delivery of data. Congestion, unstable links, packet loss, and changing delays can interrupt applications or degrade transfer performance. NetRescue explores these issues through three related components: hysteresis-based failure detection and recovery, chunk-based multipath transfer over logical paths, and a diagnostics/benchmarking dashboard. Hysteresis introduces distinct conditions for entering and leaving a state, helping reduce rapid state changes when measurements fluctuate. Multipath-transfer research studies how data chunks can be scheduled over paths with different delays and capacities. Network emulation research highlights the value of controlled, repeatable fault conditions when evaluating network behavior. NetRescue uses application-level simulations and Windows host diagnostics; it does not claim to reproduce a kernel-level network emulator or a production multipath transport implementation.

1.2 Problem Statement

Network faults can cause temporary unavailability, repeated state changes, poor transfer performance, and incomplete or corrupted data. Basic connectivity checks alone do not demonstrate recovery behavior or file integrity. NetRescue addresses this learning and evaluation problem by combining threshold-based recovery experiments, logical-path file transfer with integrity checking, and host-level network diagnostics in one project. Its goal is to make behavior observable and testable, rather than to claim production-grade routing or guaranteed performance improvement.

1.3 Scope

The intended users are networking students, researchers, IT administrators, and network engineers who need to explore failure detection, transfer integrity, and diagnostic measurements. The project covers a local frontend/backend application, Windows host diagnostics, and controlled application-level simulations. The multipath paths used in experiments are logical paths; they are not necessarily separate physical interfaces or Internet routes. Results depend on the host, destination, and test configuration. The project does not modify system routes or firewall settings and does not claim production traffic failover.

1.4 Objectives

The objectives are to evaluate threshold-based failure detection and recovery, verify file chunk reassembly and integrity, and measure network diagnostics and experiment performance using recorded outputs.

Objective ID	Objective	Verification Method
01	Evaluate hysteresis-based failure detection and recovery.	Recovery-state logs and timing measurements
02	Verify chunk reassembly and file integrity.	Chunk completion and SHA-256 comparison
03	Evaluate diagnostics and network performance.	Connectivity, DNS, route trace, latency/loss, and benchmark outputs

2. Proposed Design and Requirements

2.1 Architecture and Network Topology

NetRescue follows a local client–server architecture. A user interacts with the React/TypeScript dashboard in the browser. The dashboard sends HTTP requests to the FastAPI backend running locally. Backend modules expose recovery experiments, transfer operations, and network diagnostic/benchmark functions. The transfer experiment divides a file into chunks, schedules chunks over logical paths, reassembles the received data, and compares SHA-256 hashes. Host-level diagnostics invoke operating-system/network utilities such as ping, DNS resolution, and route tracing. Controlled fault experiments are simulated in the application and should be distinguished from measurements of the real host network.

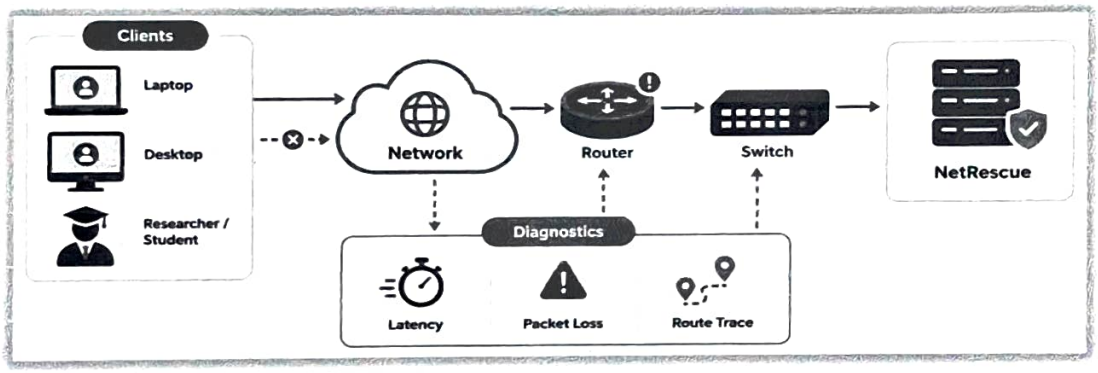


Figure 2.1. NetRescue architecture and data flow.

2.2 Tools and System Requirements

Versions below should be confirmed against the current development environment before submission. The project was developed and tested on Windows, with a Python/FastAPI backend and React/TypeScript frontend. Record the exact installed versions from the environment rather than relying on assumed versions.

Tool or Resource	Version or Specification	Purpose	
Windows	Windows development environment	Host-level network diagnostics and application execution	
Python	Installed version: confirm locally	Backend implementation	
FastAPI / Uvicorn	Installed versions: confirm locally	Backend API and local server	
React / TypeScript / Vite	Installed versions: confirm locally	Dashboard and frontend development	
SHA-256	Standard hash algorithm	Check reconstructed-file integrity	
Git / GitHub	Repository: Marvickq/NetRescue	Source control and collaboration	

2.3 Addressing and Protocol Selection

NetRescue is a local web application rather than a manually configured router topology. The API was run on 127.0.0.1:8000, and the Vite development frontend was previously run on localhost:3002. These are development endpoints; the frontend port may change if the development server selects another available port. ICMP is used by ping diagnostics, DNS is used for hostname resolution, and HTTP carries frontend/backend API requests. Direct use of TCP or UDP sockets for the file-transfer feature must be described only if confirmed from the implementation.

HTTP provides the request/response interface between the browser dashboard and FastAPI.

Device / Interface	IP / Prefix	Gateway	Role
Diagnostic destination	Record actual target used	Network-dependent	Ping/DNS/route-trace test target
Frontend dev server	Local host:3002	Not applicable	React/Vite dashboard
Backend API	127.0.0.1:8000	Not applicable	Local FastAPI service

Ping uses ICMP for reachability and round-trip measurements; DNS tools test hostname resolution; tracert reports the route observed by the host. SHA-256 is used as a content-integrity check after file reassembly. The transfer experiment models multiple logical paths in the application, so it should not be described as physical multipath TCP/UDP unless separate socket/transport implementations are confirmed.

3. Implementation and Technical Working

3.1 Methodology

The workflow begins with launching the backend and frontend. The user selects a recovery experiment, file-transfer test, or diagnostics function from the dashboard. For recovery, the experiment evaluates link-state changes against configured thresholds and records state transitions. For transfer, the file is split into chunks, scheduled across logical paths, reassembled, and checked with SHA-256. For diagnostics, the application runs or presents host-level checks and controlled benchmark results. The dashboard displays outputs for inspection and report export. Real host diagnostics and simulated experiment data are identified separately.

3.2 Configuration or Code Implementation

The backend is organized as a FastAPI application. The recovery functionality is implemented in apps/api/routers/recovery.py, with configurable failure/recovery thresholds and an optional

EWMA setting. The multipath transfer functionality includes netrescue/xferlab.py, netrescue/transfer/session.py, and dashboard components under apps/dashboard/src/multipath, including TransferCanvas.tsx and MultipathPlaygroundPage.tsx. The frontend is built with React and TypeScript using Vite. The transfer process tracks chunks and offsets, reassembles the file, and compares SHA-256 hashes. Full source code should be referenced from the project repository rather than copied in full into this section.

3.3 Protocols and Technical Working

The browser communicates with the FastAPI backend through HTTP API requests. The backend invokes the relevant recovery, transfer, or diagnostics logic and returns results for dashboard display. The diagnostics feature uses host-level network utilities for ping, DNS resolution, and route tracing. In the multipath experiment, file chunks are scheduled over simulated logical paths and then reassembled at the receiver. SHA-256 comparison checks whether the reconstructed file matches the original content. The project should not be described as implementing UDP or raw TCP sockets unless the source code confirms this.

3.4 Execution Procedure

On Windows, open PowerShell and move to the repository directory. Activate the project virtual environment and launch the backend. In a second terminal, start the frontend development server. Then open the local URL printed by Vite and use the dashboard to run a recovery experiment, transfer test, or diagnostic check. Commands used previously are shown below; adjust the repository path if it differs on the machine:

Open the Vite URL printed in the terminal. If port 8000 is already occupied, identify the existing process or choose an available port and update the frontend API configuration as needed.

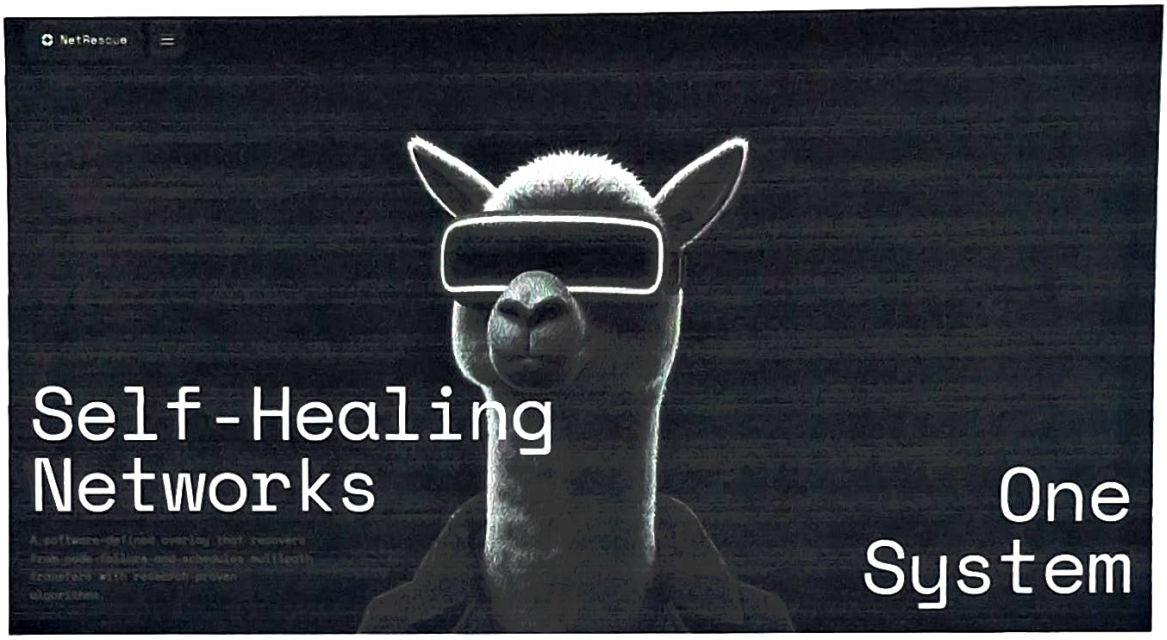


Figure 3.1. NetRescue implementation screenshot

4 Testing and Results

4.1 Test Plan

Testing should distinguish application-level test cases from real host diagnostics. The recovery test checks state transitions under configured thresholds. The transfer test checks chunk completion, reassembly, and SHA-256 equality. The diagnostics test checks that the dashboard reports outputs from connectivity, DNS, route-tracing, and benchmark functions. Record test date, configuration, output, and pass/fail status from the actual run. The project history includes a transfer verification example with 19 of 19 chunks completed and matching SHA-256 hashes; reproduce this result on the submitted version before treating it as final evidence.

Test ID	Test and Input	Expected Result	Observed Result	Status
T1	Run configured failure/recovery experiment	State transitions follow configured thresholds	Record state transitions and timing from final run	Pending verification
T2	Transfer test file across logical paths	Chunks reassemble and SHA-256 hashes match	Prior verification example: 19/19 chunks completed and hashes matched; rerun on final version	Re-test
T3	Run diagnostics and benchmark functions	Dashboard displays available diagnostic/benchmark outputs	Attach actual latency/loss, DNS, route-trace and benchmark output	Pending evidence

4.2 Results and Execution Evidence

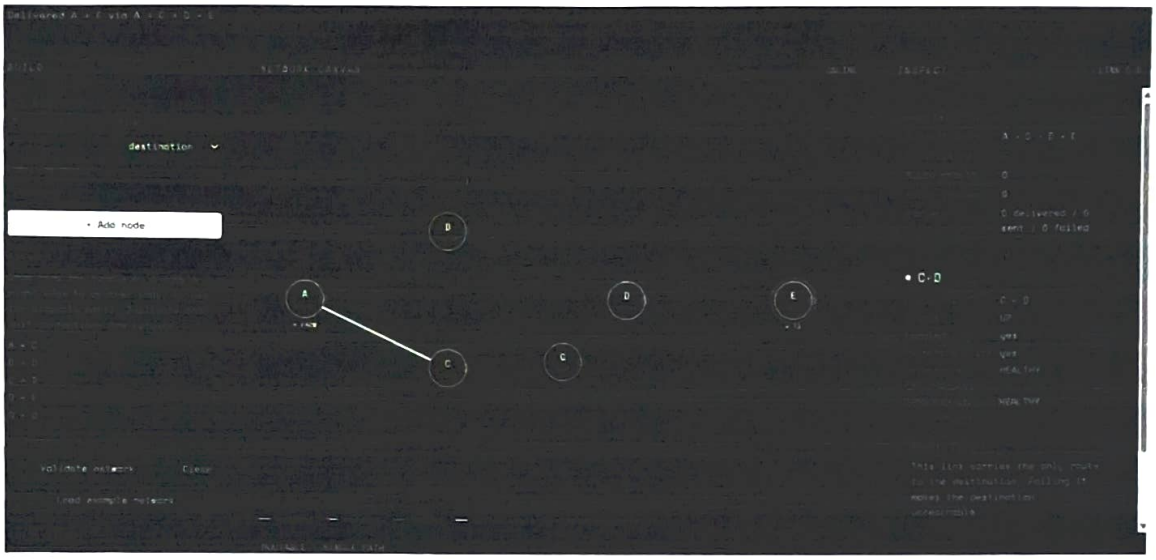


Figure 4.1. Test execution evidence

4.3 Objective Achievement

Map O1 to recovery-state and timing logs, O2 to chunk completion and SHA-256 comparison, and O3 to diagnostics and benchmark outputs. Mark each objective achieved only when the corresponding evidence from the submitted version is attached.

Objective ID	Evidence or Test ID	Outcome
O1	T1: recovery logs and timing	10s
O2	T2: chunk completion and SHA-256	It breaks into chunks as per file
O3	T3: diagnostics and benchmark outputs	Out of 10 8-9 are real is achieved

5. Discussion and Conclusion

5.1 Discussion and Comparative Analysis

NetRescue combines three capabilities that are often evaluated separately: threshold-based recovery experiments, chunk-based transfer integrity checks, and network diagnostics. The recovery component makes state transitions observable; the transfer component checks reconstruction and content integrity; the diagnostics component exposes host measurements and controlled experiment outputs. Multipath scheduling literature motivates evaluating path differences and chunk scheduling [1], while network-emulation research motivates controlled test conditions [2]. Any numerical comparison with a baseline should be included only when the same test inputs and conditions were measured. Logical-path simulation results are not evidence of real physical multipath throughput gains.

5.2 Advantages and Applications

NetRescue provides an educational environment for exploring failure detection, recovery thresholds, transfer integrity, and network measurements. It may support networking laboratory demonstrations, debugging exercises, and preliminary comparisons of scheduling or recovery configurations. The diagnostic tools can help users inspect host connectivity, DNS resolution, and routes. It is a prototype and should not be treated as a production network monitoring or automatic routing platform.

5.3 Limitations

The multipath paths are logical application-level simulations rather than guaranteed independent physical links. Controlled fault injection does not reproduce every behavior of a real network or kernel-level emulator. Measurements depend on the host, destination, and network conditions. The project does not establish that multipath transfer is always faster, and it should not be described as implementing TCP/UDP sockets, physical rerouting, or shortest-path vector algorithms unless confirmed in code. Testing on production-scale traffic and multiple physical interfaces remains outside the demonstrated scope.

5.4 Challenges and Troubleshooting

Document only problems actually encountered during development. A previously observed environment issue was that the backend port could already be occupied; the appropriate response is to identify the listening process or use a free port, then verify that the API documentation endpoint responds. For any other issue, attach the error message, root-cause

diagnosis, correction, and a successful rerun rather than recording a generic example as a historical fact.

Issue	Cause or Diagnosis	Solution and Verification
Backend port occupied (if encountered in final run)	Port 8000 may already have a listening process	Identify process or choose a free port; verify API responds
Transfer integrity mismatch (test case)	Potential missing/corrupted chunk or reassembly error	Inspect chunk records and compare SHA-256; report only if encountered
Diagnostic target unreachable (test case)	Target may be offline or blocked by network policy	Check DNS, ping, and route trace; report actual observed cause

5.5 Future Scope

Future improvements include adaptive path scheduling based on measured latency and bandwidth, testing across independent physical interfaces, richer repeatable benchmark reports, and automated regression tests for recovery and transfer behavior. A further step would be comparing the application-level simulation against a validated network-emulation environment under equivalent test conditions.

5.6 Conclusion

NetRescue brings together hysteresis-based failure detection and recovery experiments, multipath file transfer over logical paths with SHA-256 integrity checking, and network diagnostics/benchmarking. The implementation provides a way to inspect state transitions, observe chunk reassembly, and review connectivity and performance measurements. Its strongest demonstrated contribution is the integration of these learning and testing capabilities in one dashboard. The current simulation boundaries are important: results do not prove production-grade physical multipath routing or guaranteed performance improvement. The objectives should be reported as achieved only to the extent supported by final test logs and screenshots.

References

References are numbered in order of citation. Confirm the bibliographic metadata against the publisher page before submission.

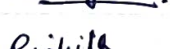
[1] "On-demand scheduling for concurrent multipath transfer using the Stream Control Transmission Protocol (SCTP)," Journal of Network and Computer Applications, 2015. [Online]. Available: https://www.sciencedirect.com/science/article/abs/pii/S108480451400215X.

[2] "An Empirical Study of NetEm Network Emulation Functionalities," in Proc. IEEE International Conference on Computer Communications and Networks (ICCCN), 2011. [Online]. Available: https://ieeexplore.ieee.org/document/6005933/. [Verify authors and page details before submission.]

AI Usage Declaration

AI tools were used for planning, technical explanations, drafting support, and coding assistance (including ChatGPT and Claude). The team is responsible for reviewing generated material against the source code, project behavior, documentation, and test evidence. Only results reproduced in the implementation and supported by logs or screenshots should be reported as verified. Add or remove tools to reflect actual use by the group.

Declaration: We take responsibility for the accuracy of this report and understand the submitted implementation and results. Sources and assistance used have been acknowledged.

Student Name	PRN	Signature
Manjiri Sakore	1262242414	
Rishita Kumar	1262242408	
Aashnaa Panda	1262241937	
Tanushree Doddamani	1262241836	

Appendices

Appendix A Code and Configuration

List the key source modules and provide repository paths or selected excerpts. Relevant modules include apps/api/routers/recovery.py, netrescue/xferlab.py, netrescue/transfer/session.py, and the dashboard multipath components under apps/dashboard/src/multipath. Do not include secrets, tokens, or credentials.

Appendix B Additional Evidence

Attach recovery experiment logs, transfer chunk summaries, SHA-256 comparisons, diagnostics screenshots, benchmark exports, and test outputs from the final submitted version. Label each item with the test ID, configuration, and date.

Appendix C Individual Contributions

Contributions below reflect the responsibilities provided by the team; add each student's full name and PRN and adjust if responsibilities changed.

Student Name and PRN	Work Completed	Evidence or Deliverable
Manjiri- 1262242414	Feature 1 and Feature 2 implementation	Recovery module; multipath transfer module
Rishita - 1262242408	Feature 3 implementation	Diagnostics and benchmarking module
Aashnaa - 1262241937	Frontend development and research	Dashboard and research material
Tanushree - 1262241836	Database management, backend integration and functional testing	Database components, test evidence and debugging records